

# Resumen Analítico del Simulador MiPS 32

**Estudiante:** Vanessa Moreno

**Profesor:** José Canache

16 de mayo de 2026

# 1. Introducción a MiPS 32

## 1.1. MIPS 32 (La Arquitectura)

Es el diseño o plano estructural de un tipo de procesador que trabaja con bloques de información de **32 bits** (cadenas de 32 unos y ceros). Se caracteriza por ser de tipo **RISC** (Conjunto de Instrucciones Reducido), lo que significa que utiliza un idioma de comandos muy corto, simple y rápido para ejecutar tareas.

## 1.2. Lenguaje Ensamblador (El Idioma)

Es el lenguaje de programación de más bajo nivel que existe antes de llegar al código máquina (los unos y ceros). Es un idioma puente que nos permite darle órdenes directas al hardware de la computadora utilizando palabras cortas en inglés (como **add** para sumar o **sub** para restablecer/restar) en lugar de escribir números binarios abstractos.

## 1.3. Registro (La Memoria Interna)

Es un espacio de almacenamiento ultra rápido pero sumamente pequeño que se encuentra **dentro** del propio procesador. MIPS 32 tiene exactamente 32 de estos registros generales (nombrados con un signo de dólar, como `\$t0` o `\$s0`). Sirven como “casillas” temporales para guardar los números exactos con los que el procesador está haciendo un cálculo en ese instante.

## 1.4. Compilar / Ensamblar (La Traducción)

Es el proceso automático que agarra el código de texto que tú escribes (ya sea el código MIPS 32 o el mismísimo código de LaTeX en Overleaf) y lo **traduce** por completo a un formato que el sistema final pueda entender y ejecutar (ya sea transformarlo a código máquina binaria para el procesador, o transformarlo a un archivo PDF estructurado).

# 2. Tabla de Registros MIPS 32

| Número de reg. | Nombre alt. | Descripción                             |
|----------------|-------------|-----------------------------------------|
| 0              | \$zero      | El valor constante cero (0).            |
| 1              | \$at        | Reservado para el ensamblador.          |
| 2–3            | \$v0 – \$v1 | Valores de resultados de funciones.     |
| 4–7            | \$a0 – \$a3 | Argumentos para las subrutinas.         |
| 8–15           | \$t0 – \$t7 | Temporales (no se conservan).           |
| 16–23          | \$s0 – \$s7 | Valores guardados (se conservan).       |
| 24–25          | \$t8 – \$t9 | Temporales adicionales.                 |
| 26–27          | \$k0 – \$k1 | Reservados para el sistema/controlador. |
| 28             | \$gp        | Puntero global (datos estáticos).       |
| 29             | \$sp        | Puntero de pila (stack pointer).        |
| 30             | \$s8 / \$fp | Valor guardado / puntero de marco.      |
| 31             | \$ra        | Dirección de retorno de la función.     |

### 3. Explicación de Operadores Básicos

A continuación se detallan los operadores fundamentales de MIPS 32 con su sintaxis estricta para el uso en la pizarra:

#### 3.1. add (Sumar registros)

Separa los operandos de derecha a izquierda. Toma el valor de dos registros, los suma y guarda el resultado en un tercer registro de destino.

- **Estructura:** `add [Destino], [Operando 1], [Operando 2]`
- **Ejemplo escrito:** `add $t0, $t1, $t2` → Suma el contenido de `$t1` más `$t2` y el resultado lo almacena en `$t0`.

#### 3.2. sub (Restar registros)

Funciona exactamente igual que la suma, pero realiza una resta matemática. El orden de los factores importa.

- **Estructura:** `sub [Destino], [Minuendo], [Sustraendo]`
- **Ejemplo escrito:** `sub $t0, $t1, $t2` → Resta `$t1` menos `$t2` y guarda la diferencia en `$t0`.

#### 3.3. addi (Sumar un número directo / Inmediato)

Se utiliza cuando no quieres sumar dos registros entre sí, sino que quieres sumarle un número entero constante (un inmediato) a un registro.

- **Estructura:** `addi [Destino], [Registro Base], [Número Entero]`
- **Ejemplo escrito:** `addi $t0, $t1, 5` → Toma el valor de `$t1`, le suma el número 5 directamente y guarda el total en `$t0`.

#### 3.4. lw (Cargar desde la RAM / Load Word)

Mueve la información desde la memoria RAM hacia un registro del procesador. Se usa para "traer" datos que guardaste previamente.

- **Estructura:** `lw [Destino], [Desplazamiento]([Registro Puntero])`
- **Ejemplo escrito:** `lw $t0, 0($s0)` → Va a la dirección de memoria de la RAM que indica `$s0` y copia ese dato dentro de `$t0`.

#### 3.5. sw (Guardar en la RAM / Store Word)

Es el proceso inverso a `lw`. Agarra el resultado de un cálculo que tienes en un registro del procesador y lo "manda a guardar" de forma segura en la memoria RAM.

- **Estructura:** `sw [Registro con el Dato], [Desplazamiento]([Registro Puntero])`
- **Ejemplo escrito:** `sw $t0, 0($s0)` → Toma el número que está en `$t0` y lo mete en la dirección de la memoria RAM señalada por `$s0`.

### 3.6. beq (Saltar si son iguales / Branch on Equal)

Compara dos registros. Si sus valores son exactamente iguales, el flujo del programa salta a una etiqueta específica.

- **Estructura:** `beq [Reg 1], [Reg 2], [Etiqueta]`
- **Ejemplo escrito:** `beq $t0, $t1, Sino` → Si `$t0 == $t1`, salta a la línea `Sino:`.

### 3.7. bne (Saltar si NO son iguales / Branch on Not Equal)

Compara dos registros. Si sus valores son diferentes, realiza el salto a la etiqueta indicada.

- **Estructura:** `bne [Reg 1], [Reg 2], [Etiqueta]`
- **Ejemplo escrito:** `bne $t0, $t1, Repetir` → Si `$t0 != $t1`, salta a la línea `Repetir:`.

### 3.8. slt (Establecer si es menor / Set on Less Than)

Compara si el primer registro es menor que el segundo. Si es verdad, activa el registro de destino poniéndolo en 1; si es falso, lo pone en 0.

- **Estructura:** `slt [Destino], [Reg 1], [Reg 2]`
- **Ejemplo escrito:** `slt $t0, $s0, $s1` → Si `$s0 < $s1`, entonces `$t0` valdrá 1. De lo contrario, valdrá 0.

### 3.9. slti (Establecer si es menor que un número directo)

Funciona igual que el `slt`, pero compara un registro contra un número entero constante (inmediato).

- **Estructura:** `slti [Destino], [Reg 1], [Número]`
- **Ejemplo escrito:** `slti $t0, $s0, 18` → Si `$s0 < 18`, `$t0` se vuelve 1. Si no, se vuelve 0.

### 3.10. j (Salto incondicional / Jump)

Fuerza al procesador a saltar directamente a una etiqueta sin hacer ninguna comparación previa.

- **Estructura:** `j [Etiqueta]`
- **Ejemplo escrito:** `j FinPrograma` → Salta directamente a la sección `FinPrograma:`.

### 3.11. `sll` (Desplazamiento lógico a la izquierda / Shift Left Logical)

Desplaza los bits de un registro a la izquierda. Mover los bits  $N$  posiciones equivale matemáticamente a multiplicar el número por  $2^N$ .

- **Estructura:** `sll [Destino], [Reg Base], [Cantidad de bits]`
- **Ejemplo escrito:** `sll $t0, $s0, 2` → Desplaza los bits de `$s0` dos posiciones a la izquierda (lo multiplica por 4) y guarda el resultado en `$t0`.

### 3.12. `li` (Cargar valor inmediato / Load Immediate)

Es el comando más sencillo para asignar un valor a una variable. Mete un número directo dentro de un cajón de memoria.

- **Estructura:** `li [Destino], [Número Entero]`
- **Ejemplo escrito:** `li $t0, 10` → Borra lo que haya en `$t0` y le mete el número 10 directamente.

## Estructura General de un Programa

Un programa en MIPS 32 se escribe en archivos de texto plano con la extensión `.s` (requerida por simuladores como SPIM o MARS) y se divide rígidamente en dos secciones principales:

### 1. Declaración de Datos (`.data`)

Esta sección está identificada con la directiva `.data`. Aquí se reservan y declaran los nombres de las variables que utilizará el programa. El espacio de almacenamiento para estos datos se asigna directamente en la memoria principal (RAM).

### 2. Código del Programa (`.text`)

Esta sección se identifica con la directiva `.text` y contiene las instrucciones físicas que ejecutará el procesador.

- **Punto de partida:** La ejecución comienza obligatoriamente en la etiqueta llamada `principal`: (o `main`:).
- **Punto final:** El programa no se detiene solo; al finalizar la lógica, se debe usar explícitamente la llamada al sistema `exit` para cerrarlo de forma segura.

### 3. Comentarios

Para documentar el código en la pizarra o en el editor, se utiliza el símbolo `#`. Todo lo que se escriba después de este símbolo en la misma línea será ignorado por el ensamblador.

## Plantilla Base de un Programa en Ensamblador

A continuación se presenta el esquema básico y obligatorio que debe seguir cualquier programa escrito en MIPS 32:

```
# =====
# Nombre del programa: Plantilla.s
# Descripción: Esquema básico de un programa en MIPS 32
# =====

        .data      # Sección de datos: las variables van aquí
                   # (Ejemplo: mensajes de texto, arreglos, etc.)

        .text      # Sección de código: las instrucciones van aquí

principal:      # Indica el inicio exacto de la ejecución
                # Aquí va tu código (add, sub, lw, etc.)

                # Fin del programa:
                # Se debe invocar la llamada al sistema 'exit'

# Nota: Dejar siempre una línea en blanco al final del archivo.
```